Case Study Report

School of Future Tech ITM Skills University

Case Study Title: Programming Logic Puzzle Game

By: Mohammed Saad Mulla
Roll Number: 150096725112

Index

1. Introduction to the Case Study
2. Problem Statement / Case Background (Abstract)
3. Case Study Design
4. Methods & Algorithms / Technology Applied
5. Implementation Details and Snapshots
6. Results and Conclusion
7. References

---

1. Introduction to the Case Study

This case study is about a **Programming Logic Puzzle Game** — a browser based game where players answer coding questions to escape through 3 rooms. The idea is simple, you are shown a code snippet and asked what it will output. Get it right, you move forward. Get it wrong, you try again.

---

2. Problem Statement / Case Background (Abstract)

Background

Many first year students pass theory exams but cant actually read or write code properly. The gap between theory and practice is very common. Most online platforms are either too advanced or too dry for beginners. There was a need for something simple, fun and focused only on the basics.

Abstract

This project is a **Programming Logic Puzzle Game** built with HTML5, CSS3 and JavaScript. It has 3 rooms with 3 questions each. Players must answer all 3 correctly to unlock the next room. Each question shows a code snippet and the player has to predict the output.

Features include a 30 second countdown timer, a hint system that costs 5 points, a max score of 90 and localStorage to save the last score. The design is cyberpunk themed with neon colours and animations to make it look intresting and engaging.

---

3. Case Study Design

The game is a single page web app with multiple screens that switch based on what the player does. The flow is as follows:

Loading Screen — An animated progress bar fills up and then the start screen appears. Done using `setInterval`.

Start Screen — Shows the game title and a button to enter. If the player played before, their saved score is shown.

Game Screen — Three room boxes are shown. Room 1 is open, rooms 2 and 3 are locked. As the player clears rooms, the next one unlocks.

Puzzle Panel — Slides up from the bottom when a room is clicked. Shows the question, code, 4 options, hint button and a timer.

Result Screen — Shows final score and grade after all rooms are cleared.

The project has 3 files:

- `index.html` — structure of all screens
- `style.css` — design and animations
- `script.js` — all game logic and data

---

4. Methods & Algorithms / Technology Applied

Programming Concepts Used

Variables — `var` is used to store score, current room, current question, hint status and timer.

Arrays and Objects — All 9 questions are stored in a `rooms` array. Each room is an object with a name and a questions array inside it.

Functions — The game has 11 functions, each doing one specific job like `startGame`, `loadQ`, `checkAnswer`, `solveRoom` etc.

if/else — Used to check if answer is correct, to prevent score going negative, and to assign grade on result screen.

for loops — Used to reset rooms, create answer buttons dynamically, and disable buttons after answering.

setInterval / setTimeout — `setInterval` runs the loading bar and countdown timer. `setTimeout` adds small delays between screen transitions.

DOM Manipulation — `getElementById` finds and updates HTML elements. `createElement` builds buttons. `classList.add/remove` toggles CSS classes for animations.

localStorage — Saves and reads the player's score so it persists even after closing the browser.

Technology Stack

Technology	Purpose
HTML5	Page structure and all screens
CSS3	Design, animations, glow effects
JavaScript	Game logic, scoring, timer, DOM
Google Fonts	Orbitron and Share Tech Mono fonts
localStorage API	Saving score across sessions

CSS Techniques

- **Glassmorphism** — `backdrop-filter: blur(18px)` for frosted glass cards

- **Neon glow** — `text-shadow` and `box-shadow` with cyan colour
- **Animations** — `@keyframes` for floating blobs, pulsing rooms, blinking timer
- **Slide-up panel** — `transform: translateY(100%)` hides panel below screen, adding class `open` slides it up
- **Responsive** — `@media` query for smaller screens

---

5. Implementation Details and Snapshots

Question Topics

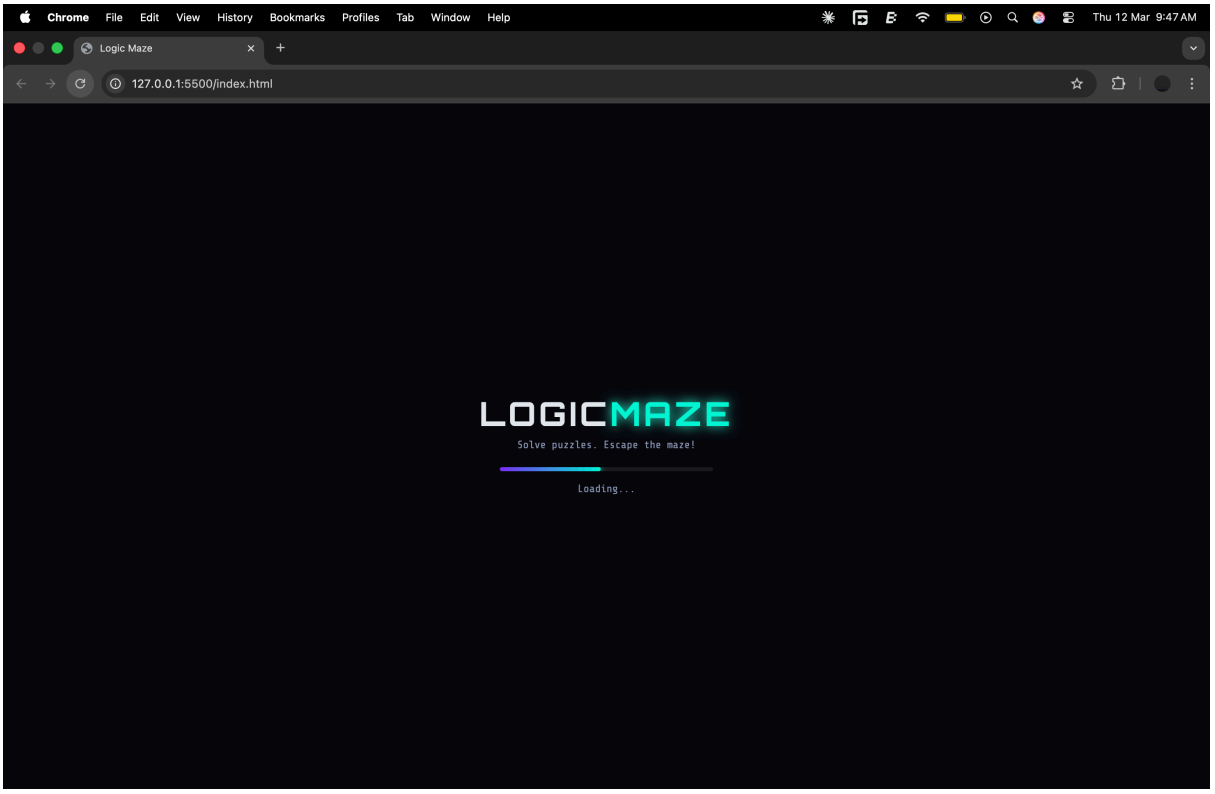
Room	Topics Covered
Room 1 — Entry Gate	if/else, arithmetic, for loop
Room 2 — Array Hall	array indexing, functions, .length
Room 3 — Final Door	number patterns, && operator, accumulation loop

Scoring

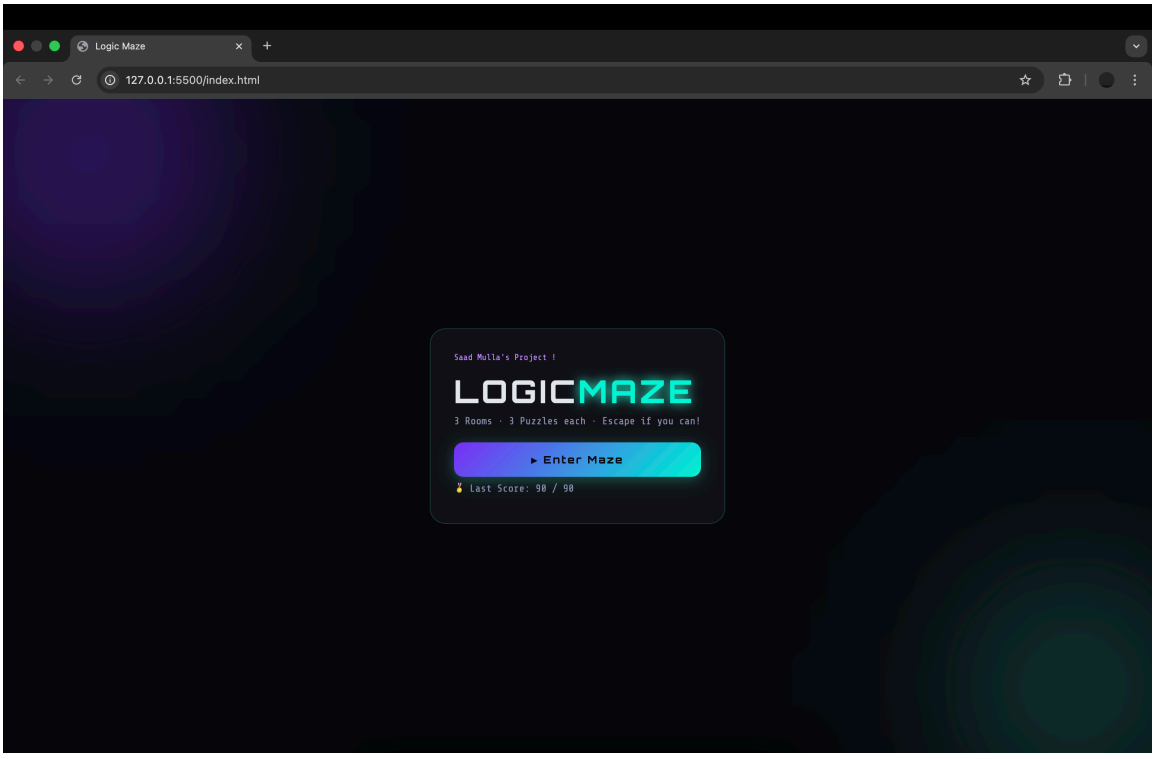
Action	Points
Correct answer	+10
Using hint	-5
Wrong answer	0, panel closes
Maximum score	90

Snapshots to Include

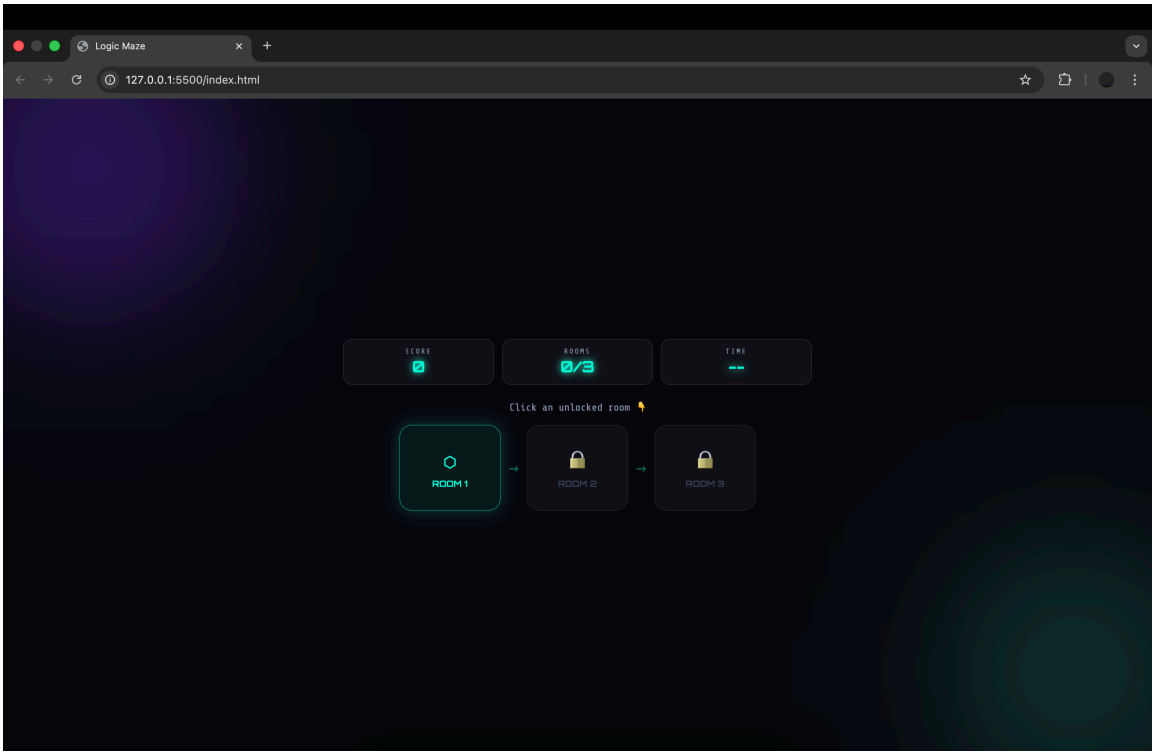
- Loading screen with progress bar



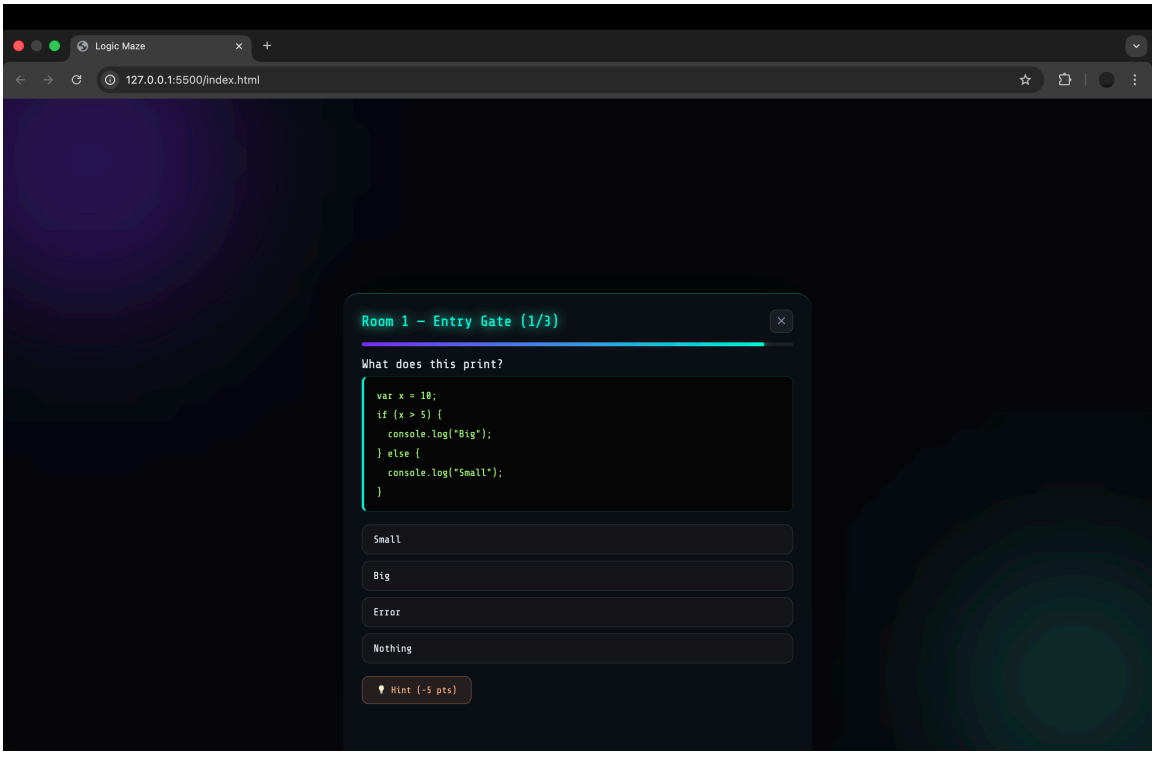
- Start screen



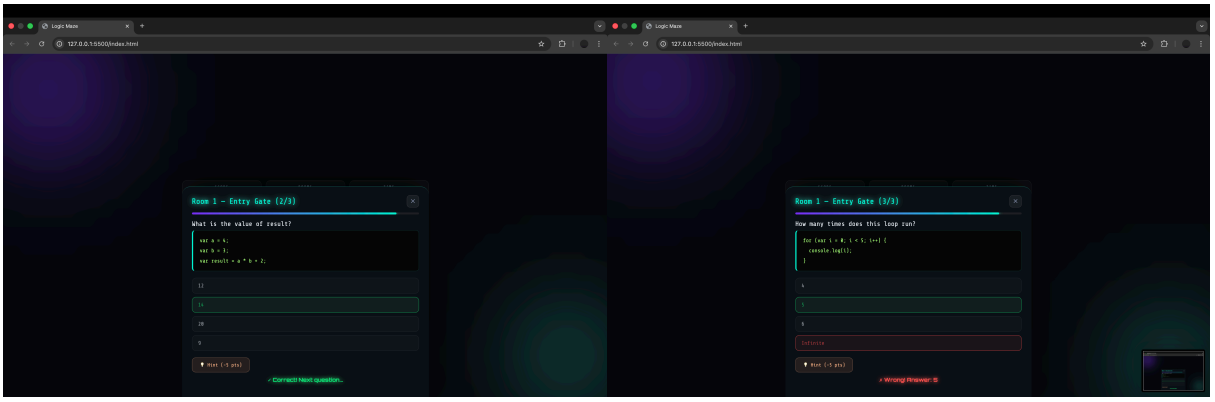
- Game screen with 3 rooms



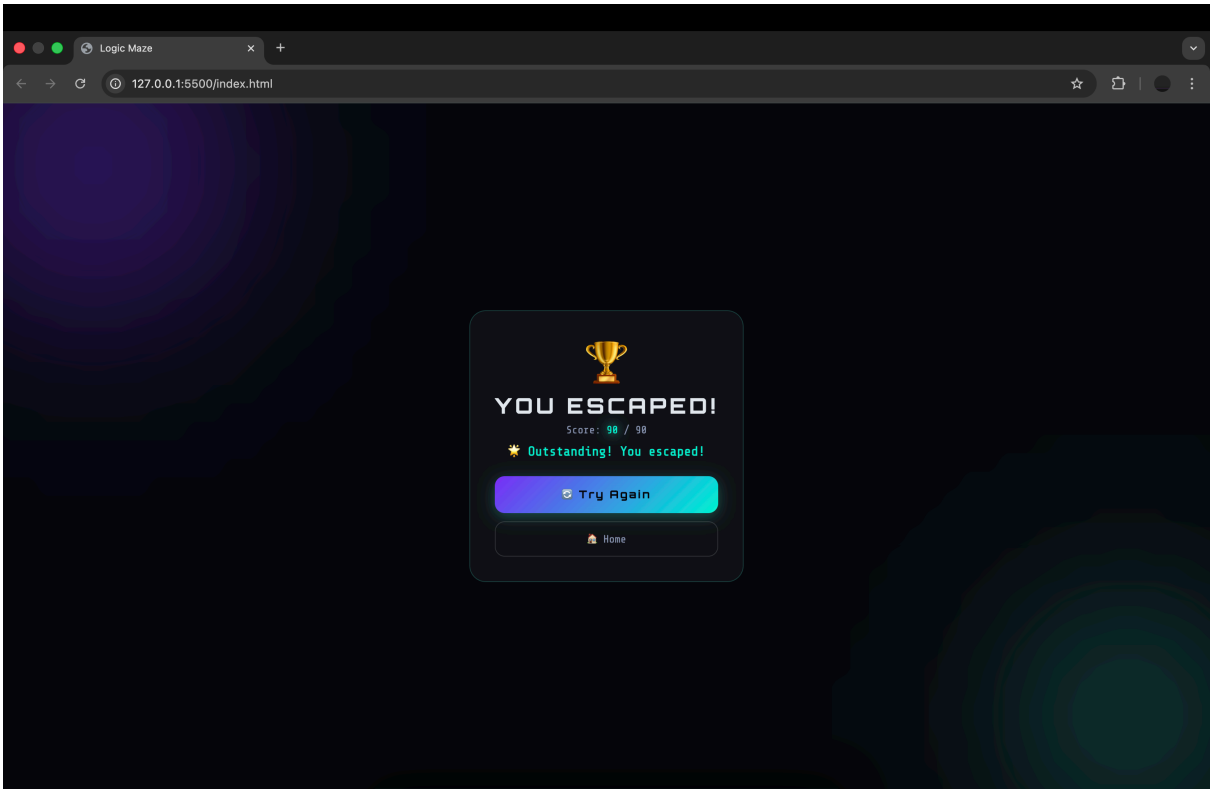
- Puzzle panel with question and options



- Correct and wrong answer feedback



- Result screen with final score



---

6. Results and Conclusion

Findings

After testing the complete game the following was observed:

- The timer adds pressure and stops the player from guessing randomly
- The hint system works well, players think twice before using it since it costs points
- Room based progression gives a feeling of achievement after each room
- localStorage saves score correctly and shows it on the next visit
- Score cap at 90 works and no overflow issues occured

Conclusion

This project builds a working **Programming Logic Puzzle Game** that makes basic programming concepts more interactive for beginners. It only uses HTML, CSS and JavaScript with no external libraries which keeps everything simple and understandable.

The game covers real topics from the first year syllabus — conditions, loops, arrays and functions — and presents them in a format that makes the student actually think instead of just read. This project shows that even basic web technologies can be used to create something with real educational value.

---

7. References

- MDN Web Docs — JavaScript, DOM, localStorage
- MDN Web Docs — CSS animations, flexbox, backdrop-filter
- Google Fonts — Orbitron, Share Tech Mono
- W3Schools — HTML5, CSS3, JavaScript reference